1. PROJECT DESCRIPTION

1.1 INTRODUCTION

Fuel is one of the most essential resources in modern life, powering vehicles, generators, and a wide range of industrial machinery. With the growing dependence on fuel, its proper management has become a critical challenge in both developed and developing economies. Fuel theft, unauthorized usage, and leakage account for substantial losses every year in the transportation industry, logistics operations, and even in small-scale fuel-dependent applications. Apart from economic losses, these issues also reduce operational efficiency, increase downtime, and create security risks.

Traditionally, fuel monitoring has been carried out using manual gauges or periodic inspections, where an operator physically checks the fuel level. While this approach has been in practice for decades, it is highly inefficient and often inaccurate. Manual readings are prone to human error and can be manipulated, making them unreliable in environments where accountability and transparency are necessary. Similarly, mechanical gauges provide only a rough estimate of fuel levels without offering any data logging, real-time notifications, or theft detection features. As industries are moving toward smart automation, relying on such outdated methods poses serious challenges.

The introduction of embedded systems, IoT (Internet of Things), and cloud-based platforms has opened new opportunities for developing smart fuel monitoring systems. By combining sensors, microcontrollers, and wireless communication technologies, it is now possible to monitor fuel levels with high precision and provide real-time alerts to users.

The **Digital Fuel Level Indicator and Reserve Alarm System** is designed as a solution to these issues. Using an ESP32 microcontroller, ultrasonic sensor, GSM communication, and IoT cloud integration, the system continuously monitors fuel levels, identifies unusual consumption patterns, and alerts the user in case of abnormalities. The proposed solution is cost-effective, scalable, and easy to implement in vehicles, generators, and industrial fuel tanks. With its ability to provide local and remote monitoring, the system significantly enhances fuel security.

Additionally, the system's real-time data analytics enable predictive maintenance and efficient fuel management. This innovation not only reduces operational costs but also promotes transparency and sustainability in fuel usage.

1.2 EXISTING SYSTEM

In most practical cases today, fuel monitoring is still managed by traditional methods. Vehicles and fuel tanks are often equipped with manual dipsticks or float-type gauges, which provide an approximate reading of the fuel level. Operators physically inspect these readings and log them for recordkeeping. While such systems are low-cost and easy to use, they come with several limitations. First, they are highly vulnerable to tampering, which means they cannot reliably prevent or detect theft. Second, these systems only provide information when manually checked, meaning there is no real-time monitoring or automated alert system.

Some advanced fleets and industries use digital gauges that display the fuel level electronically. While this improves visibility, they are still local systems and do not offer remote monitoring or data storage. In many cases, even these electronic gauges are not precise enough to detect small changes in fuel levels, such as gradual leakages or partial thefts. Moreover, these systems often lack integration with communication technologies such as GSM or cloud-based IoT platforms, which means users cannot access fuel data remotely or receive timely alerts.

Another common limitation of existing systems is their inability to store historical data. Without proper data logging, operators cannot analyze fuel consumption trends or identify irregularities over time. This makes it difficult to differentiate between legitimate fuel usage and unauthorized activity. Furthermore, industries relying on fuel for critical operations often suffer from delayed responses to theft or leakage, resulting in increased costs and reduced productivity.

Although there are commercially available fuel monitoring solutions in the market, many of them are expensive, require complex installation, or are not scalable for smaller applications like individual vehicles or small-scale generators. As a result, there is a need for a simple, affordable, and effective system that bridges these gaps by offering **real-time monitoring, theft detection, alert notifications, and cloud integration** in a user-friendly design.

1.3 PROPOSED SYSTEM

The **Digital Fuel Level Indicator and Reserve Alarm System** is proposed as an innovative solution to overcome the drawbacks of traditional monitoring methods. This system leverages IoT, GSM communication, and embedded systems to provide an automated, real-time, and highly reliable fuel monitoring process. The central unit of the system is the **ESP32 microcontroller**, which collects data

from an ultrasonic sensor placed on the fuel tank. The sensor measures the distance between the fuel surface and the top of the tank, translating this measurement into accurate fuel levels.

Unlike existing systems, the proposed solution does more than just display the fuel level. The ESP32 processes the sensor readings to identify patterns of normal consumption and detect abnormal drops, which may indicate theft, leakage, or unauthorized usage. Once such a situation is identified, the system instantly triggers the **GSM module** to send an SMS notification to the registered user, ensuring that they are immediately aware of the situation regardless of their location.

To further enhance its usability, the system integrates with an **IoT cloud platform**. This allows fuel data to be uploaded in real-time, providing remote access via smartphones or computers. The cloud dashboard not only displays the current fuel level but also maintains a record of historical data, enabling users to analyze consumption trends and make informed decisions about fuel usage. A **local LCD display** is also included to provide on-site monitoring, ensuring that users can check the fuel status directly at the tank.

The system is designed to be cost-effective, scalable, and easy to install. It requires minimal maintenance and can be deployed across different applications such as vehicles, industrial tanks, and power generators.

1.3.1 Hardware Specification

PROCESS : INTEL® CORE™ I9-14900K 3.20 GHZ

RAM : 16 GB

HARD DISK : 1 TB

Ultrasonic sensor

LCD

IOT

SMS / GCM

MICROCONTROLLER ESP32

1.3.2 Software Specification

ARDUINO IDE, Embedded c Programming language, Arduino embedded cloud

2. LOGICAL DEVELOPMENT

The development of the **Digital Fuel Level Indicator and Reserve Alarm System** is rooted in the growing demand for accurate, reliable, and automated methods of fuel monitoring. The logical progression of the system design begins with identifying the limitations of traditional fuel monitoring methods and then introducing modern technologies to overcome those limitations in a structured manner.

The starting point of the design process was the recognition that conventional fuel monitoring systems—such as dipsticks, mechanical gauges, or manual inspections—do not provide real-time information and are highly prone to errors or tampering. These weaknesses motivated the shift toward an electronic and automated approach. The primary objective was to build a system that could **monitor fuel levels continuously, detect anomalies instantly, and alert the user without delay**.

The next logical step was selecting suitable hardware components. An **ultrasonic sensor** was chosen to measure the fuel level because it is non-intrusive, accurate, and reliable. Unlike float sensors that come into direct contact with fuel and degrade over time, ultrasonic sensing offers durability and precision. Mounting the sensor on top of the tank ensures that the readings remain unaffected by fuel impurities or environmental conditions.

The **ESP32 microcontroller** was selected as the processing unit due to its versatility, low power consumption, and built-in support for Wi-Fi and Bluetooth, which make it ideal for IoT applications. The ESP32 processes the input from the ultrasonic sensor and applies logic to differentiate between normal fuel consumption and abnormal, sudden drops in the fuel level. This processing stage represents the "decision-making" core of the system, where rules and thresholds are applied to classify events as routine usage, reserve level, or possible theft/leakage.

Once the decision is made, the logical flow directs the system toward communication. A **GSM module** was incorporated to deliver SMS alerts instantly to the registered user. This ensures that notifications are received even when internet access is unavailable. At the same time, the ESP32 transmits data to an **IoT cloud dashboard**, which allows continuous remote monitoring, historical data logging, and trend analysis. This dual approach—local alerts via SMS and global access via cloud—provides robustness and user convenience. To enhance user interaction, a **local LCD display** was included, offering immediate visual feedback about the fuel level and system status. Finally, a regulated power

supply ensures that all components operate reliably under varying conditions, completing the logical framework of the system.

In summary, the logical development of the proposed system follows a clear path: identifying the problem, replacing manual processes with automation, selecting precise sensing technology, integrating smart microcontroller processing, enabling dual communication (GSM + IoT), and providing both local and remote access. This logical flow guarantees that the system not only detects and alerts theft or abnormal fuel activity but also supports efficient and transparent fuel management.

2.1 DATA FLOW DIAGRAM

2.1.1 DATA FLOW DIAGRAM – LEVEL 0

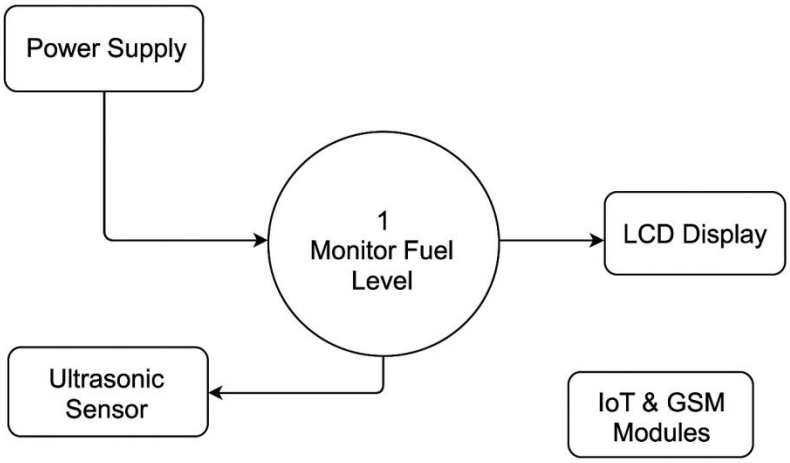

Figure 2.1.1 Data Flow Diagram – Level 0 (DFD 0)

The Level 0 Data Flow Diagram represents the high-level overview of the Digital Fuel Level Indicator and Reserve Alarm System. It shows the system as a single process, "Monitor Fuel Level," which interacts with external entities such as the **Power Supply**, **Ultrasonic Sensor**, **LCD Display**, and **IoT/GSM Modules**. Input is received from the power unit and ultrasonic sensor, processed by the ESP32 microcontroller, and output is provided through the LCD and IoT/GSM communication modules. This diagram highlights the overall system boundary, showing how external entities exchange data with the central process in a simple, structured manner.

2.1.2 DATA FLOW DIAGRAM – LEVEL 1

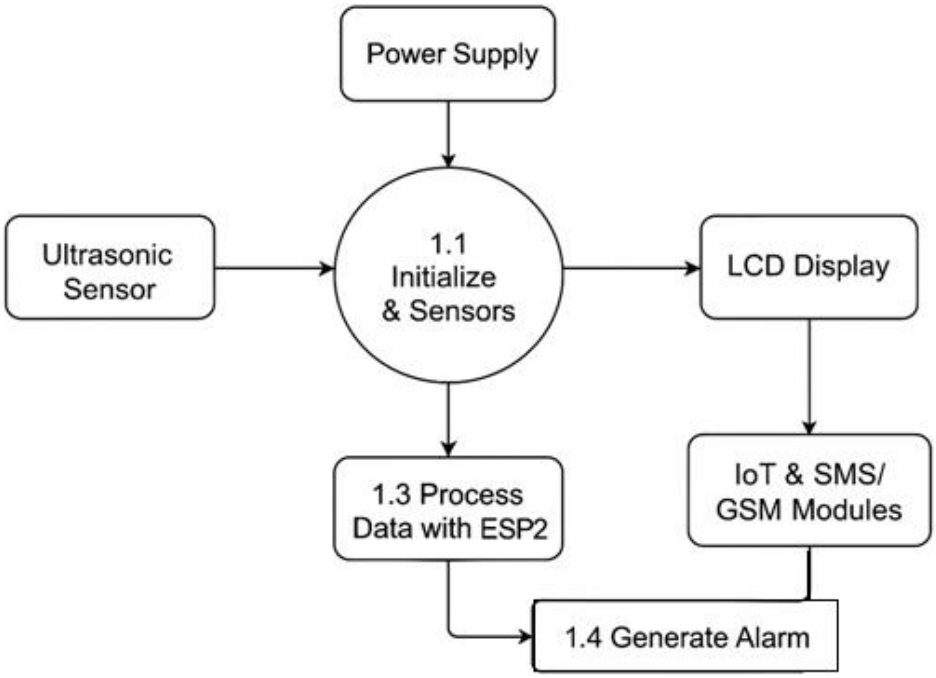

Figure 2.1.2 Data Flow Diagram – Level 1 (DFD 1)

The Level 1 Data Flow Diagram breaks down the main process into four detailed sub-processes: **Initialize & Sensors**, **Process Data with ESP32**, **Generate Alarm**, and **Update Displays**. The ultrasonic sensor provides continuous fuel level readings, which are initialized and passed to the ESP32 for analysis. The ESP32 then checks for abnormalities such as sudden drops or reserve levels. Based on the results, the system either generates an alarm via IoT/GSM modules or updates the LCD and cloud dashboard for real-time monitoring. This decomposition provides a clearer picture of internal data flows and logical operations within the system. The system ensures data consistency by performing periodic calibration checks on the ultrasonic sensor, maintaining measurement accuracy over time. Communication between the ESP32 and peripheral modules follows a structured data protocol, minimizing transmission errors and ensuring reliable updates. Power management is also integrated into the design, where the microcontroller enters low-power mode during idle states to enhance overall energy efficiency. Additionally, the IoT/GSM module not only sends alert notifications but also logs event data to the cloud, allowing remote access and historical analysis of fuel usage trends. The display unit provides intuitive visual indicators such as bar graphs and color-coded warnings, enabling users to easily interpret fuel levels and system status. Furthermore, the overall system architecture promotes scalability, allowing additional sensors or communication modules to be integrated in the future without requiring major hardware or software

modifications. This approach ensures flexibility, reliability, and efficiency in continuous monitoring and control of the fuel management system.

2.2 BLOCK DIAGRAM

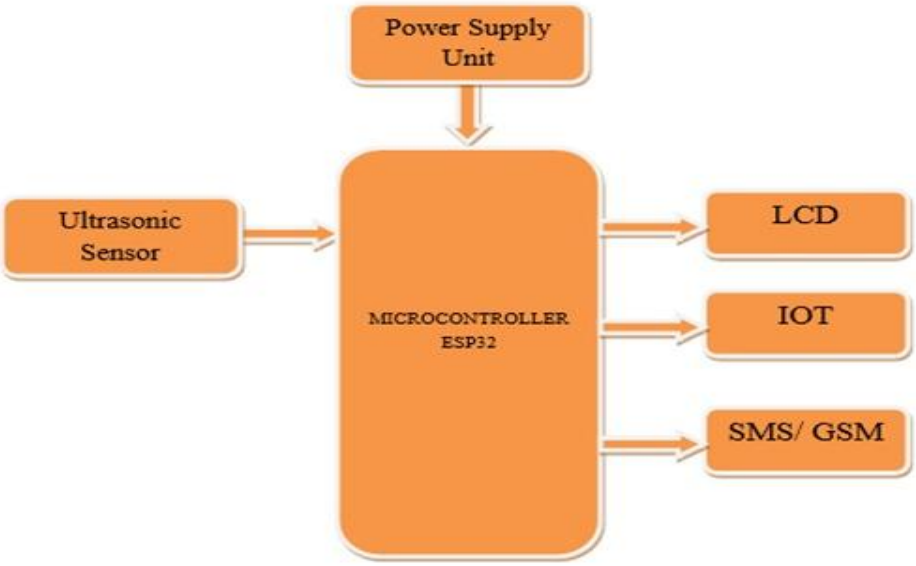

Figure 2.2.1 Block Diagram

2.3 CIRCUIT DIAGRAM

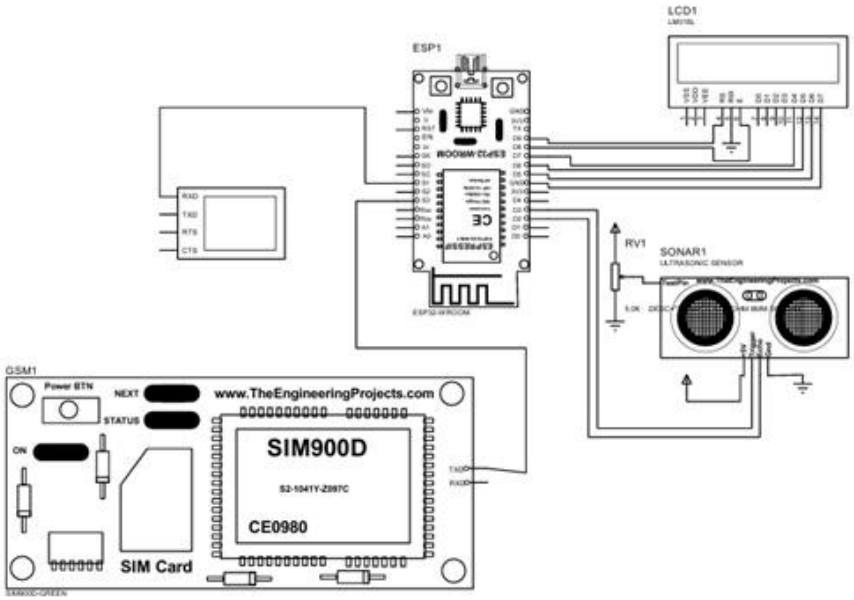

Figure 2.3.1 Circuit Diagram

2.4 WORKFLOW DIAGRAM

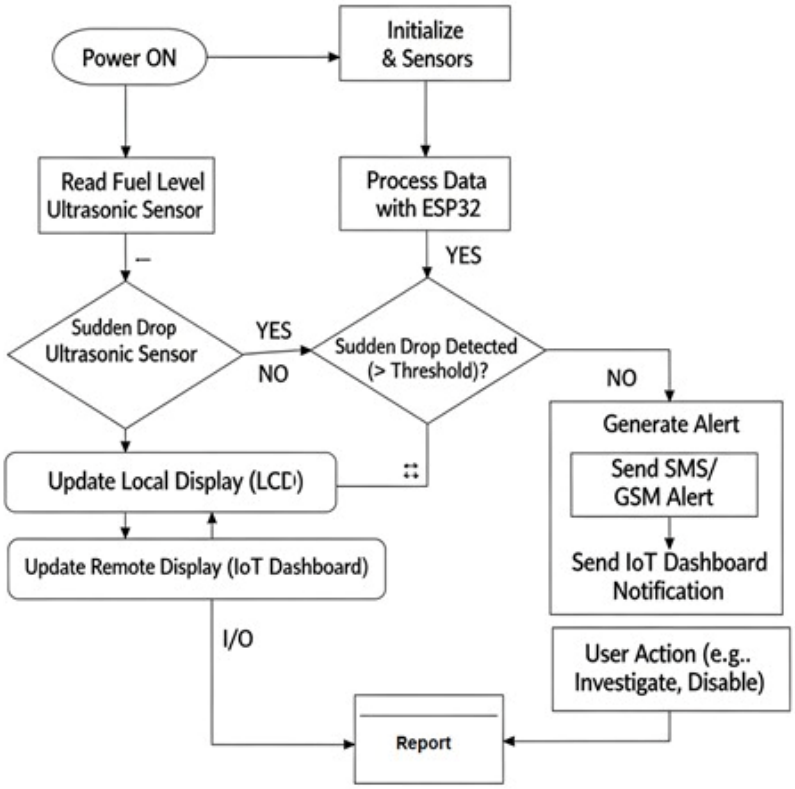

Figure 2.4.1 Workflow Diagram

2.5 ARCHITECTURE DIAGRAM

The architecture diagram illustrates the integration of the ESP32 microcontroller with key hardware components for smart fuel monitoring. The ultrasonic sensor reads the fuel level and sends data to the ESP32, which processes this information. The ESP32 simultaneously updates a local LCD, communicates real-time fuel data to an IoT cloud platform, and controls a GSM module to send SMS alerts during fuel theft or reserve conditions. All components receive regulated power from the supply unit, ensuring stable operation. This seamless data flow delivers visible on-site monitoring, remote access, and timely alert generation for reliable and efficient fuel management The ESP32 acts as the central controller, coordinating data exchange among all connected modules. The ultrasonic sensor continuously measures the fuel level and provides accurate input to the ESP32 for processing. The GSM module enables instant SMS alerts whenever abnormal conditions or fuel theft are detected. Through IoT

cloud integration, the system allows remote access and real-time monitoring of fuel data from any location.

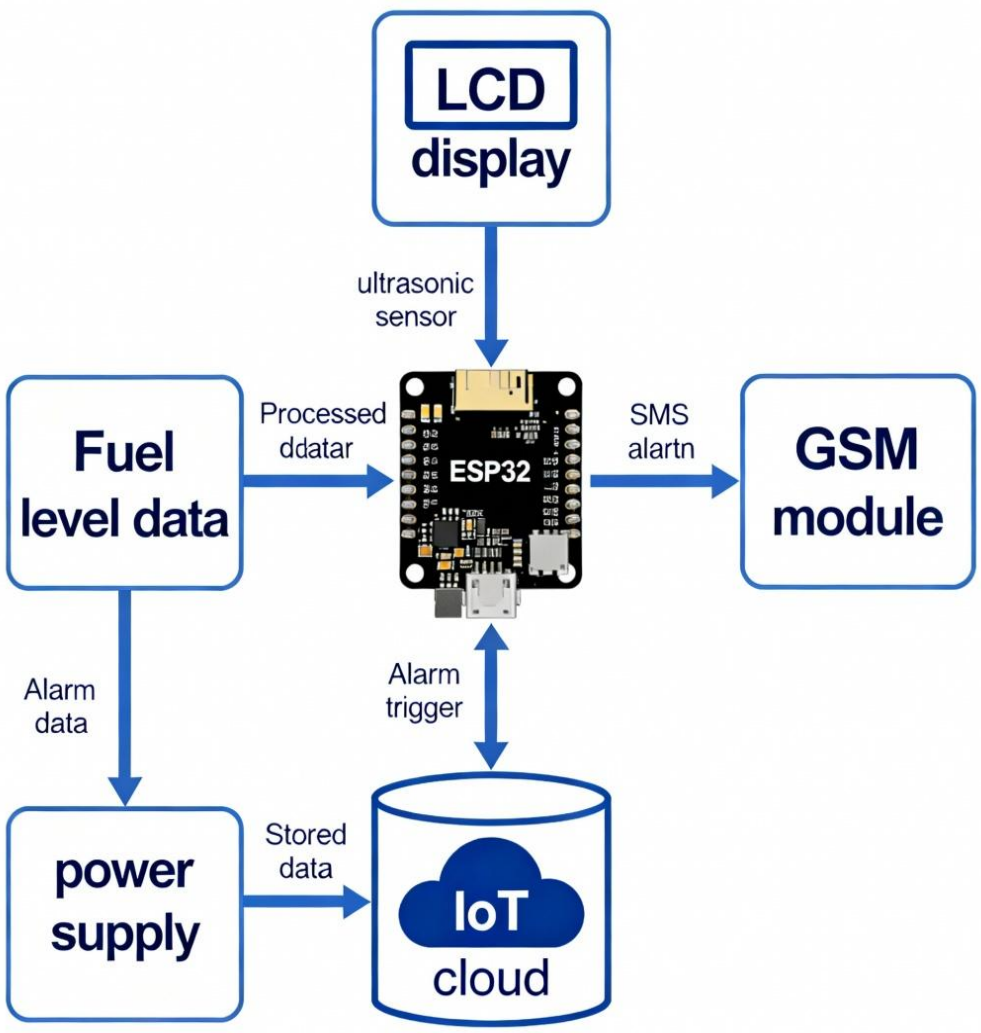

Figure 2.5.1 Architecture Diagram

3. DATABASE DESIGN

The database design of the **Digital Fuel Level Indicator and Reserve Alarm System** is a critical component of the IoT-based monitoring platform. Its primary purpose is to store, manage, and retrieve information related to fuel levels, device metadata, alerts, and user data efficiently. Since the system continuously collects real-time data from the ultrasonic sensor, the database must handle frequent updates and provide reliable access for both local and remote visualization.

At the heart of the database is the **Fuel Data Table**, which records time-stamped readings of fuel levels for each device. Each entry contains the **device ID**, measured fuel level, and the timestamp of recording. This table serves as the backbone for monitoring normal fuel consumption, detecting sudden drops, and analyzing historical trends. By maintaining a chronological record, the system can detect anomalies such as theft or leakage and provide actionable insights.

The **Device Table** holds metadata about each monitoring unit, including device ID, associated fuel tank or vehicle, and its capacity. This ensures that each reading can be accurately linked to its source, which is especially important in fleet management scenarios where multiple devices are active simultaneously. A one-to-many relationship is maintained between the **User Table** and the **Device Table**, as a single user may manage multiple fuel tanks or vehicles. The **User Table** contains user credentials, contact information, and alert preferences, enabling secure access to the cloud dashboard and SMS notifications via the GSM module.

An **Alerts Table** is implemented to log abnormal events, such as sudden drops in fuel level or reaching reserve thresholds. Each alert entry includes the device ID, alert type, timestamp, and status (read/unread). This ensures that all abnormal events are tracked and that users can review them retrospectively.

For accountability and system management, an optional **System Logs Table** can record events like login activity, configuration changes, and system updates. This enhances security and allows administrators to monitor usage patterns.

The database is designed following normalization principles to avoid redundancy and ensure efficient storage. Indexing is applied to frequently queried fields such as timestamps and device IDs to

improve performance, especially when generating reports or visualizing long-term trends. Cloud platforms like **Firebase, MySQL, or MongoDB** can be used depending on scalability requirements, enabling seamless integration with the ESP32 microcontroller and IoT dashboard.

In summary, the database design provides a **robust, organized, and scalable solution** for storing fuel level data, device information, user credentials, and alert history. By supporting real-time data collection, historical analysis, and efficient alert management, it forms the backbone of the proposed system, enhancing decision-making, fuel security, and overall operational efficiency.

3.1 DATA DICTIONARY

A **Data Dictionary** is an organized collection of all data elements, their types, descriptions, and relationships within the database. For the **Digital Fuel Level Indicator and Reserve Alarm System**, the data dictionary defines the structure of the database used to store fuel readings, alerts, device information, and user details. This ensures data consistency, integrity, and clarity for system design, development, and maintenance.

Table 3.1.1 User Table

Field Name	Data Type	Description
User_ID	Integer (PK)	Unique identifier for each registered user
Name	Varchar(25)	Full name of the user
Email	Varchar(20)	Email address for login and notifications
Contact_No	Varchar(15)	Mobile number for receiving SMS alerts
Password	Varchar(20)	Encrypted password for user authentication

Table 3.1.2 Device Table

Field Name	Data Type	Description
Device_ID	Integer (PK)	Unique identifier for each device
User_ID	Integer (FK)	References the user owning the device
Tank_Name	Varchar(30)	Name or identifier of the fuel tank/vehicle
Tank_Capacity	Float	Maximum capacity of the fuel tank in liters

Table 3.1.3 Fuel Data Table

Field Name	Data Type	Description
Record_ID	Integer (PK)	Unique identifier for each fuel reading
Device_ID	Integer (FK)	References the device from which data is collected
Fuel_Level	Float	Current measured fuel level in liters
Timestamp	DateTime	Date and time of the fuel reading

Table 3.1.4 Alerts Table

Field Name	Data Type	Description
Alert_ID	Integer (PK)	Unique identifier for each alert
Device_ID	Integer (FK)	References the device triggering the alert
Alert_Type	Varchar(25)	Type of alert (Theft, Reserve, Leakage)
Alert_Time	DateTime	Time when the alert was generated
Status	Varchar(20)	Read/Unread status of the alert

Table 3.1.5 System Logs Table (Optional)

Field Name	Data Type	Description
Log_ID	Integer (PK)	Unique identifier for each system event
User_ID	Integer (FK)	References the user performing an action
Activity_Type	Varchar(50)	Type of activity (Login, Configuration Change, etc.)
Timestamp	DateTime	Time of the system event

3.2 Tables Design

The **Tables Design** forms the backbone of the database for the **Digital Fuel Level Indicator and Reserve Alarm System**, ensuring organized storage of all relevant data for efficient fuel monitoring, theft detection, and alert management. Each table has been designed to store specific types of data while maintaining relationships that enable seamless integration with the IoT dashboard, GSM alerts, and local display.

3.2.1 User Table

The **User Table** is designed to maintain information about all registered users of the system. Each user is uniquely identified by a User_ID, which serves as the primary key. Other fields include the user's name, email, contact number, and password. The contact number is particularly important as it is used by the GSM module to send SMS alerts. By linking users to multiple devices through a one-to-many relationship, the system can accommodate fleet management scenarios where a single user monitors several fuel tanks or vehicles. This table ensures secure access to the cloud dashboard and provides essential information for sending timely notifications.

3.2.2 Device Table

The **Device Table** stores metadata about each monitoring unit, including Device_ID, associated User_ID, tank or vehicle name, and tank capacity. Each fuel sensor device is uniquely tracked to ensure accurate mapping of fuel readings and alerts. This table plays a crucial role in linking sensor readings from the **Fuel Data Table** to specific devices and users. Proper device management ensures that historical fuel trends can be analyzed per tank or vehicle and that alerts are accurately assigned.

3.2.3 Fuel Data Table

The **Fuel Data Table** is the core repository of fuel readings. It stores Record_ID, Device_ID, fuel level, and timestamp for each measurement. This design allows continuous logging of fuel levels, enabling real-time monitoring, historical analysis, and detection of abnormal drops. The table ensures accurate and chronological data storage to support the cloud dashboard, reports, and analytics. Furthermore, the structured data format facilitates seamless integration with IoT platforms and external

databases. This enhances system scalability and ensures reliable performance across diverse monitoring applications.

3.2.4 Alerts Table

The **Alerts Table** records all abnormal fuel events, including sudden drops, reserve-level warnings, or potential theft. Fields include Alert_ID, Device_ID, alert type, timestamp, and status (read/unread). This table enables users to review past alerts, respond to incidents promptly, and maintain accountability in fuel management.

3.2.5 System Logs Table

Optionally, a **System Logs Table** captures administrative and system activities such as logins, configuration changes, and updates. This table improves security and auditability by keeping track of all significant events.

The table design collectively ensures **structured, scalable, and efficient data management**, supporting automated fuel monitoring, real-time alerts, and insightful analytics.

3.3 RELATIONSHIP DIAGRAM

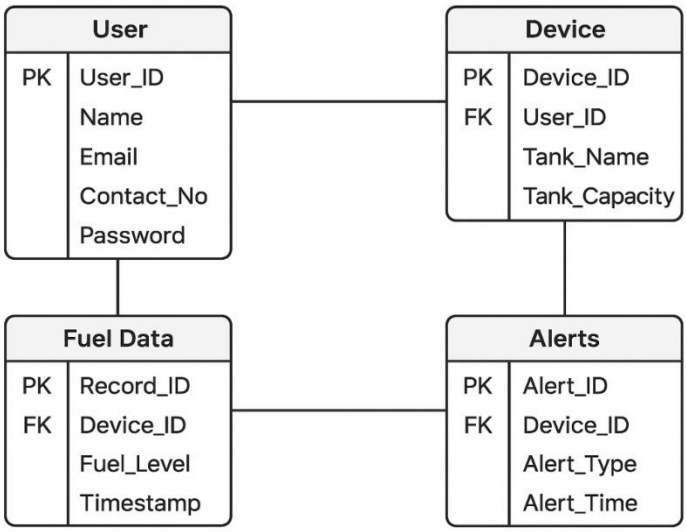

Figure 3.3.1 Relationship Diagram

4. PROGRAM DESIGN

4.1 HARDWARE SPECIFICATION

PROCESSOR : INTEL® CORE™ I9-14900K 3.20 GHZ

The **Intel® Core™ i9-14900K processor** with a base clock speed of 3.20 GHz is a powerhouse in terms of computational capabilities. Designed for intensive workloads, it offers exceptional multi-core performance, making it ideal for tasks like gaming, video editing, 3D rendering, and other processor-intensive applications. Its high clock speed and modern architecture ensure efficient data processing, quick task execution, and minimal latency.

4.1.1 RAM

With **16 GB of RAM**, the system provides ample memory for seamless multitasking. This capacity supports the smooth operation of modern software, gaming, and professional applications like video editing tools or machine learning frameworks. It ensures that switching between applications remains fluid, without system slowdowns.

4.1.2 HARD DISK

The **1 TB hard disk** offers a significant amount of storage space to accommodate the operating system, essential software, media files, and other data. While traditional hard drives provide large storage capacities, combining this with an SSD can greatly enhance system performance, particularly in boot times and file transfers.

4.2 ULTRASONIC SENSOR

An ultrasonic sensor is a device that uses sound waves to measure the distance to an object. It emits an ultrasonic pulse and measures the time it takes for the pulse to reflect back from the object. By calculating the time delay, it determines the distance between the sensor and the object. The speed of sound in air is used to convert the time delay into a distance measurement. Ultrasonic sensors are commonly used in robotics, automotive parking systems, and industrial automation.They are effective for detecting objects regardless of color, texture, or lighting conditions. However, their accuracy can be affected by temperature, humidity, and the surface shape of the target.

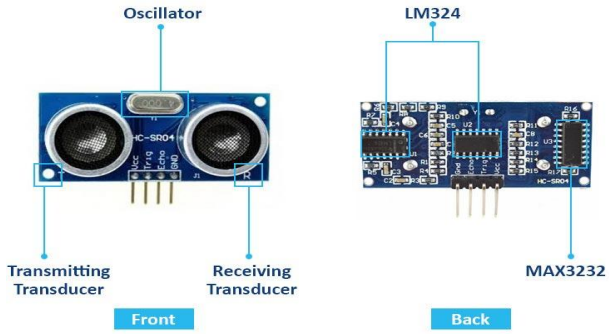

Figure 4.2.1 Ultrasonic Sensor

4.2.1 WORKING PRINCIPLE OF ULTRASONIC SENSOR

An ultrasonic sensor works by emitting high-frequency sound waves, typically beyond the range of human hearing, and measuring the time it takes for these waves to travel to an object and bounce back. The sensor consists of a transmitter that sends out an ultrasonic pulse and a receiver that detects the reflected sound waves. When the sensor emits the sound wave, it travels through the air until it encounters an object, which reflects the wave back to the sensor. The sensor measures the time it takes for the sound to travel to the object and return. Using the speed of sound in air, the sensor calculates the distance to the object. The formula used is:

$$\text{Distance} = \frac{\text{Speed of Sound} \times \text{Time}}{2}$$

The factor of 2 accounts for the round trip of the sound wave. By processing the time and the known speed of sound, the sensor can determine the precise distance between itself and the object. This non-contact measurement method makes ultrasonic sensors valuable in a wide range of applications, such as obstacle detection, distance measurement, and level sensing.

4.2.2 PIN CONFIGURATION OF ULTRASONIC SENSOR

VCC	Power supply pin (typically +5V).
Trig (Trigger):	Sends a 10 µs HIGH pulse to start the ultrasonic wave.
Echo	Outputs a HIGH signal with duration proportional to the return

4.2.3 SPECIFICATIONS OF ULTRASONIC SENSORS

Operating Voltage: 5V DC

Max Current Consumption: ~15 mA during normal operation (up to 50 mA during transmission)

Operating Frequency: 40 kHz

Range: 2 cm to 400 cm (0.02 m to 4 m)

Accuracy: ±3 mm

Resolution: 1 cm (0.01 m)

Beam Angle: About 15 degrees

Response Time: Around 10 ms

Output Type: Digital (Echo pin outputs a HIGH signal, the pulse width corresponds to distance)

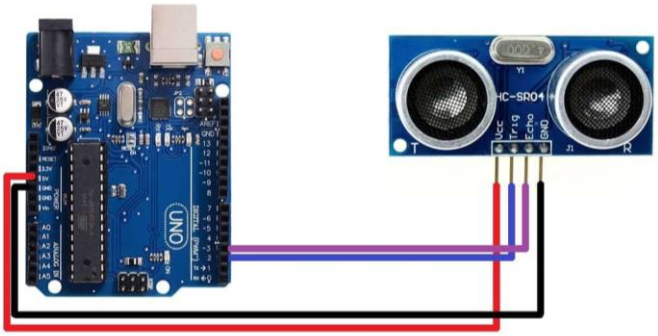

Figure 4.2.2 Specifications of Ultrasonic Sensor

4.2.4 APPLICATIONS OF ULTRASONIC SENSORS

- Robotics: Used in robots for obstacle detection, mapping surroundings, and navigation.
- Automotive: In parking sensors for distance measuring to prevent collisions.
- Industrial Automation: For distance measurement in conveyor belts, material handling, and robotic arms.
- Water Level Measurement: Used in tanks, wells, and reservoirs to monitor water levels.
- Gesture Recognition: Detecting hand or body gestures in interactive systems.

- Medical: Ultrasonic sensors are used in medical devices for measurements and imaging, like in ultrasound imaging.

4.2.5 SP32 MICROCONTROLLER

The ESP32 is an exceptionally versatile and powerful microcontroller designed for a wide range of Internet of Things (IoT) applications. Developed by Espressif Systems, it integrates advanced features, including built-in Wi-Fi and Bluetooth capabilities, which facilitate seamless connectivity to the internet and other devices. At its core, the ESP32 is powered by a dual-core Tensilica LX6 processor, capable of running at speeds up to 240 MHz, enabling it to handle multiple tasks simultaneously. This makes it suitable for complex applications that require significant processing power, such as real-time data analytics and control systems.

With 520 KB of SRAM and the ability to support external SPI flash memory of up to 16 MB, the ESP32 can accommodate extensive programming and data logging requirements. Its rich set of peripherals includes up to 34 programmable GPIO pins, ADC (Analog-to-Digital Converters) with 12-bit resolution, and DAC (Digital-to-Analog Converters) for generating audio signals or other analog outputs. The presence of various communication protocols, such as SPI, I2C, and UART, allows for easy integration with a multitude of sensors and devices, enhancing its applicability across various projects.

Power management is another strength of the ESP32, featuring several power-saving modes that are particularly beneficial for battery-operated devices. These modes allow the microcontroller to enter deep sleep states, significantly reducing power consumption when the device is idle. Furthermore, robust security features, including hardware encryption (AES, SHA-2), secure boot, and flash encryption, ensure secure communication and data integrity, which are essential for many IoT applications where data security is paramount.

The ESP32 is widely supported by various development environments, including the Arduino IDE and Espressif's own IoT Development Framework (ESP-IDF), which provide extensive libraries and examples to accelerate the development process. Its applications span diverse fields, from smart home devices that automate lighting and security systems to wearable technology for fitness tracking and health monitoring. The microcontroller is also used in environmental monitoring systems that measure temperature, humidity, and air quality, as well as in industrial automation for monitoring equipment performance and controlling automated processes. In the realm of robotics, the ESP32 serves as a central control unit, facilitating wireless communication and sensor integration for complex robotic systems.

In conclusion, the ESP32 stands out as a highly capable and adaptable microcontroller, combining performance, versatility, and extensive connectivity features in a compact package. Its dual-core architecture and comprehensive set of peripherals make it an excellent choice for a wide variety of applications in the ever-expanding field of IoT. Whether for simple hobbyist projects or sophisticated industrial solutions, the ESP32 continues to gain popularity among developers and engineers for its performance, ease of use, and robust feature set, ultimately paving the way for innovative technological advancements in numerous domains.

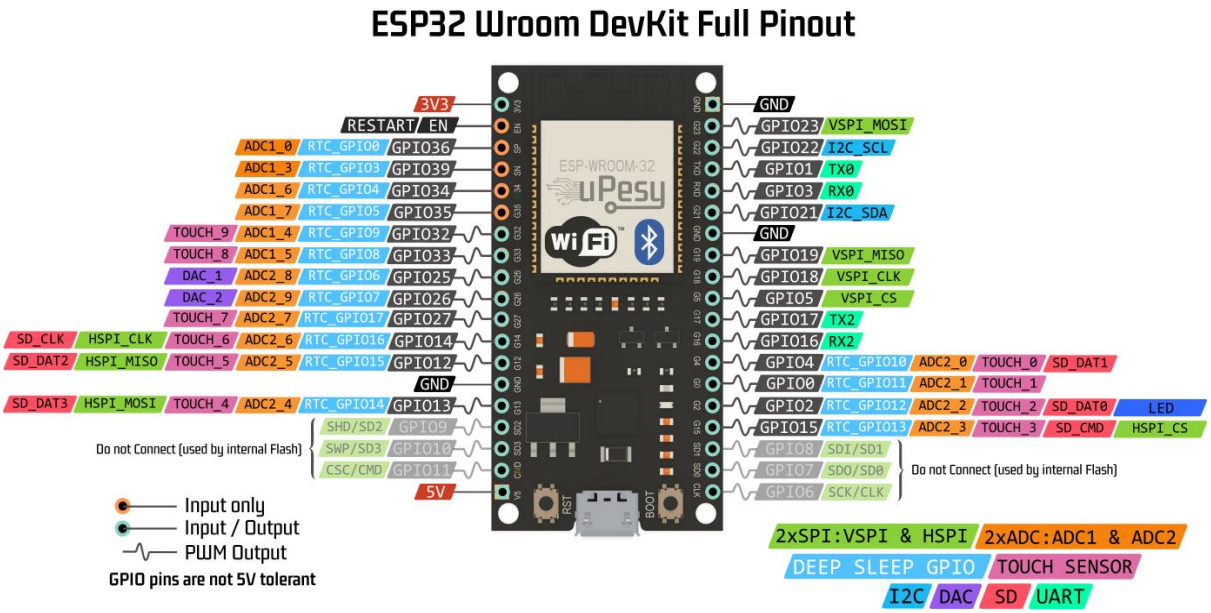

Figure 4.2.3 ESP32 Wroom DevKit Full Pinout

4.2.6 ESP32 PIN FUNCTIONS

- GPIO Pins: Up to 34 configurable General Purpose Input/Output pins for versatile applications. These pins can be programmed for digital input or output, making them suitable for interfacing with a variety of sensors and actuators.
- ADC: Up to 18 channels with 12-bit resolution for accurate analog signal measurement. This feature allows the ESP32 to read varying voltage levels from sensors, providing precise data for applications like environmental monitoring.
- DAC: 2 channels with 8-bit resolution for generating analog outputs. The DAC can be used for audio applications or to control devices that require variable voltage levels.

- Power Pins: V5 for 5V input, V3 for 3.3V output, and multiple GND pins. These power pins facilitate connections to external devices and ensure that the ESP32 operates within its required voltage range.
- I2C Interface: SDA and SCL pins for connecting I2C devices. The I2C protocol allows multiple devices to share the same bus, simplifying the wiring and connections needed for complex projects.
- SPI Interface: Standard pins for MISO, MOSI, SCK, and CS for SPI communication. SPI is a high-speed interface ideal for connecting fast peripherals, such as sensors and memory cards.
- UART Interface: TX and RX pins for serial communication. These pins enable the ESP32 to communicate with other devices over a serial interface, useful for debugging and data transmission.
- Touch Sensor Pins: Configurable pins for touch-sensitive input. This feature allows for the development of user interfaces that can detect touch input, enhancing interactivity in applications.
- RTC Functions: Pins for real-time clock operations and wake-up triggers. These functions are essential for applications requiring time-based operations, even during low power modes.
- Programming/Debugging: EN pin for chip enable and IO0 for boot mode selection during programming. These pins are crucial for loading new firmware and debugging the ESP32 during development.

4.2.7 ESP32 Specifications

- Microprocessor: Dual-core Tensilica LX6 processor with up to 240 MHz clock speed. This architecture allows for efficient handling of multiple tasks simultaneously, making it suitable for demanding applications.
- Memory: 520 KB of SRAM and support for up to 16 MB of external SPI flash memory. The ample memory capacity enables the execution of complex algorithms and the storage of extensive data.
- Wireless Connectivity: Integrated Wi-Fi (IEEE 802.11 b/g/n) and Bluetooth (Classic and BLE). This dual connectivity enhances the flexibility of the ESP32 in IoT applications, allowing seamless communication with various devices.

- GPIO Pins: Up to 34 General Purpose Input/Output pins for various applications. These pins can serve multiple functions, such as analog input, PWM output, or digital signaling, making the ESP32 adaptable to different project needs.
- Analog Features: 12-bit ADC with up to 18 channels and 2 channels of 8-bit DAC. The high-resolution analog features are ideal for applications requiring precise signal measurement and generation.
- Power Supply: Operates within a voltage range of 3.0V to 3.6V. This range allows the ESP32 to be powered by various sources, including batteries and regulated power supplies, enhancing its usability in portable applications.
- Power Management: Multiple power-saving modes, including deep sleep. These modes significantly reduce power consumption, making the ESP32 suitable for battery-operated devices and energy-efficient applications.
- Operating Temperature: Functional range from -40°C to 85°C. This wide temperature range ensures reliable performance in diverse environmental conditions, from industrial applications to outdoor deployments.
- Security Features: Hardware encryption (AES, SHA-2), secure boot, and flash encryption. These security measures help protect sensitive data and ensure the integrity of the device, which is critical in IoT applications.
- Development Support: Compatible with Arduino IDE, Espressif ESP-IDF, and other frameworks. The extensive development support, along with a rich set of libraries and resources, simplifies the programming process and accelerates project development.

4.3 LIQUID CRYSTAL DISPLAY

A liquid crystal display (LCD) is a flat panel display, electronic visual display, or video display that uses the light modulating properties of liquid crystals. Liquid crystals do not emit light directly. LCDs are available to display arbitrary images (as in a general-purpose computer display) or fixed images which can be displayed or hidden, such as preset words, digits, and 7-segment displays as in a digital clock. They use the same basic technology, except that arbitrary images are made up of a large number of small pixels, while other displays have larger elements. An LCD is a small low cost display. It is easy to interface with a micro-controller because of an embedded controller (the black blob on the back of the board). This controller is standard across many displays (HD 44780) which means many micro-

controllers (including the Arduino) have libraries that make displaying messages as easy as a single line of code.

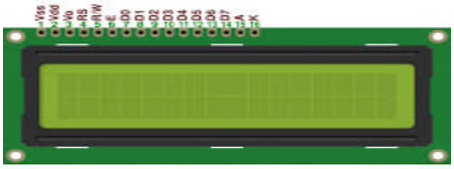

Figure 4.3.1 Liquid Crystal Display

LCDs are used in a wide range of applications including computer monitors, televisions, instrument panels, aircraft cockpit displays, and signage. They are common in consumer devices such as video players, gaming devices, clocks, watches, calculators, and telephones, and have replaced cathode ray tube (CRT) displays in most applications. They are available in a wider range of screen sizes than CRT and plasma displays, and since they do not use phosphors, they do not suffer image burn-in. LCDs are, however, susceptible to image persistence.

4.3.1 16X2 LCD SPECIFICATIONS

- Display Format: 16 characters per line, 2 lines total.
- Character Size: 5x8 pixels for standard characters.
- Dimensions: Approximately 80mm x 36mm x 13mm.
- Interface: Parallel (4-bit or 8-bit mode).
- Supply Voltage: Typically 5V DC.
- Current Consumption: Around 1.5 mA at 5V.
- Backlight: LED backlight (3.3V to 5V).
- Temperature Range: 0°C to 70°C operating, -20°C to 80°C storage.
- Response Time: Under 10 ms.
- Mounting: PCB or breadboard compatible.
- Character Set: Standard ASCII with custom character support.

4.3.2 I2C (Inter-Integrated Circuit)

The I2C connection for an LCD display enables efficient and simplified interfacing with microcontrollers, like the ESP32, by requiring only two data lines: SDA (data) and SCL (clock). This setup significantly reduces the number of pins needed, compared to traditional parallel connections, allowing other I/O pins on the microcontroller to remain free for additional sensors or peripherals. The

I2C interface is controlled by a small I2C module on the back of the LCD, typically using a chip like the PCF8574, which converts the I2C signals into parallel signals the LCD can interpret.

Each device on the I2C bus has a unique address (e.g., 0x27 for many LCD modules), letting the microcontroller communicate with specific devices even if there are multiple I2C components on the same bus.

The power (VCC) and ground (GND) lines complete the connections, and the data transfer is synchronous, meaning the microcontroller (master) sends commands in sync with the LCD (slave). Commands like positioning the cursor, clearing the display, or updating text are all handled through simple I2C libraries that streamline programming. This makes I2C ideal for projects that require organized wiring and efficient communication, especially in setups where pin limitations and space are considerations.

4.4 IOT

4.4.1 INTERNET OF THINGS

The Internet of Things (IoT) is the network of devices such as vehicles, and home appliances that contain electronics, software, actuators, and connectivity which allows these things to connect, interact and exchange data. The IoT involves extending Internet connectivity beyond standard devices, such as desktops, laptops, smartphones and tablets, to any range of traditionally dumb or non-internet-enabled physical devices and everyday objects. Embedded with technology, these devices can communicate and interact over the Internet, and they can be remotely monitored and controlled.

Figure 4.4.1 Representation of IoT

The definition of the Internet of things has evolved due to convergence of multiple technologies, real-time analytics, machine learning, commodity sensors, and embedded systems. Traditional fields of embedded systems, wireless sensor networks, control systems, automation (including home and building automation), and others all contribute to enabling the Internet of things.

4.4.2 HOW IOT WORKS

An IoT ecosystem consists of web-enabled smart devices that use embedded processors, sensors and communication hardware to collect, send and act on data they acquire from their environments. IoT devices share the sensor data they collect by connecting to an IoT gateway or other edge device where data is either sent to the cloud to be analyzed or analyzed locally. Sometimes, these devices communicate with other related devices and act on the information they get from one another. The devices do most of the work without human intervention, although people can interact with the devices -- for instance, to set them up, give them instructions or access the data. The connectivity, networking and communication protocols used with these web-enabled devices largely depend on the specific IoT applications deployed.

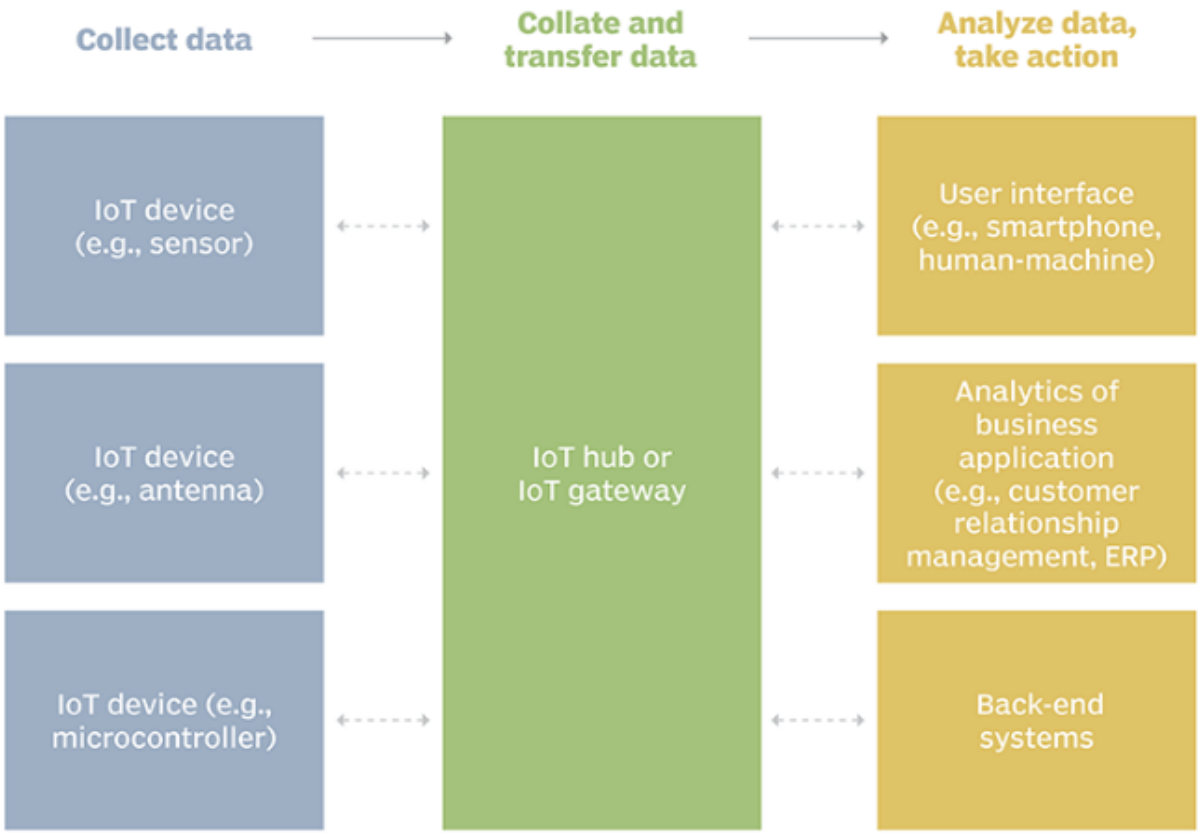

Figure 4.4.2 Example of an IOT system

4.4.3 BENEFITS OF IOT

The internet of things offers a number of benefits to organizations, enabling them to:

- monitor their overall business processes;
- improve the customer experience;
- save time and money;
- enhance employee productivity;
- integrate and adapt business models;
- make better business decisions; and
- Generate more revenue.

IoT encourages companies to rethink the ways they approach their businesses, industries and markets and gives them the tools to improve their business strategies.

4.4.4 IOT SECURITY AND PRIVACY ISSUES

The internet of things connects billions of devices to the internet and involves the use of billions of data points, all of which need to be secured. Due to its expanded attack surface, IoT security and IoT privacy are cited as major concerns.

One of the most notorious recent IoT attacks was Mirai, a botnet that infiltrated domain name server provider Dyn and took down many websites for an extended period of time in one of the biggest distributed denial-of-service (DDoS) attacks ever seen. Attackers gained access to the network by exploiting poorly secured IoT devices.

Because IoT devices are closely connected, all a hacker has to do is exploit one vulnerability to manipulate all the data, rendering it unusable. And manufacturers that don't update their devices regularly -- or at all -- leave them vulnerable to cybercriminals.

Additionally, connected devices often ask users to input their personal information, including names, ages, addresses, phone numbers and even social media accounts -- information that's invaluable to hackers.

However, hackers aren't the only threat to the internet of things; privacy is another major concern for IoT users. For instance, companies that make and distribute consumer IoT devices could use those devices to obtain and sell users' personal data.

Beyond leaking personal data, IoT poses a risk to critical infrastructure, including electricity, transportation and financial services.

4.4.5 IOT APPLICATION AREAS

Near Field Communication (NFC), Radio frequency Identification (RFID), Machine-to-Machine Communication (M2M) & Vehicle-to-Vehicle Communication (V2V) are the technologies by which IoT is being implemented exponentially. It is assumed that more than 50 billion IoT devices will be connected through internet. It is going to change human life, working style, entertaining ways and many more. IoT have many Applications Areas and domain of these application are increasing day by day.

There are ample of applications of IoT as follow:

- Smart Cities
- Building & Home automation
- Environmental Monitoring
- Automotive Industry
- Smart Retail
- Smart Agriculture
- Smart Industry
- Energy Management
- Healthcare Monitoring

4.5 EMBEDDED SYSTEM

An embedded system is one kind of a computer system mainly designed to perform several tasks like to access, process, store and also control the data in various electronics-based systems. Embedded systems are a combination of hardware and software where software is usually known as firmware that is embedded into the hardware. One of its most important characteristics of these systems is, it gives the o/p within the time limits. Embedded systems support to make the work more perfect and convenient. So, we frequently use embedded systems in simple and complex devices too. The applications of embedded systems mainly involve in our real life for several devices like microwave, calculators, TV remote control, home security and neighborhood traffic control systems, etc. Embedded systems can be found in both consume electronics and industrial machines. They are designed to operate reliably with limited resources such as memory and processing power. Advancements in technology continue to make embedded systems faster, smarter, and more energy-efficient.

An embedded system is integration of hardware and software, the software used in the embedded system is set of instructions which are termed as a program. The microprocessors or microcontrollers used in the hardware circuits of embedded systems are programmed to perform specific tasks by following the set of instructions. These programs are primarily written using any programming software like Proteus or Lab-view using any programming languages such as C or C++ or embedded C. Then, the program is dumped into the microprocessors or microcontrollers that are used in the embedded system circuits.

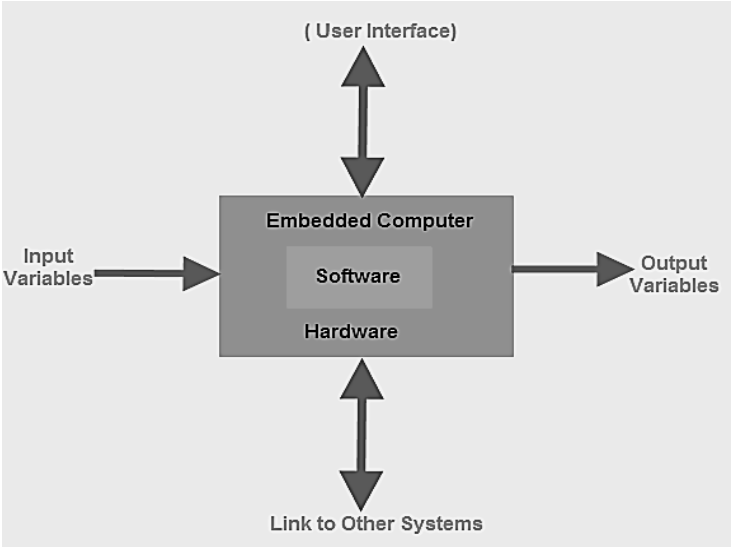

Figure 4.5.1 Embedded System

4.5.1 Embedded System Classification

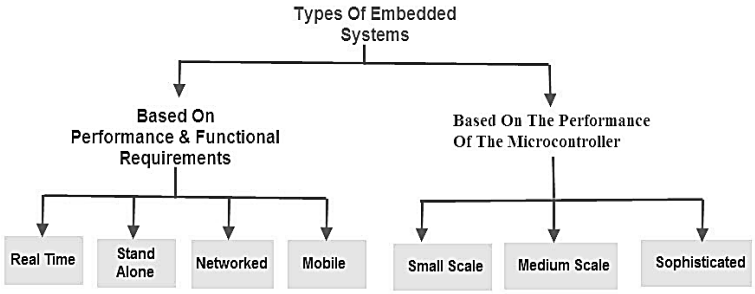

Figure 4.5.2 Embedded System Classification

Embedded systems are primarily classified into different types based on complexity of hardware & software and microcontroller (8 or 16 or 32-bit). Thus, based on the performance of the microcontroller, embedded systems are classified into three types such as:

- Small scale embedded systems
- Medium scale embedded systems
- Sophisticated embedded systems

Further, based on performance and functional requirements of the system embedded system classified into four types such as:Real time embedded systems

- Standalone embedded systems
- Networked embedded systems
- Mobile embedded systems

4.5.2 Embedded System Hardware

An embedded system uses a hardware platform to perform the operation. Hardware of the embedded system is assembled with a microprocessor/microcontroller. It has the elements such as input/output interfaces, memory, user interface and the display unit. Generally, an embedded system comprises of the following

- Power Supply
- Memory
- Processor
- Timers
- Output/Output circuits
- Serial communication ports

- SASC (System application specific circuits)

4.5.3 Embedded System Software

The software of an embedded system is written to execute a particular function. It is normally written in a high-level setup and then compiled down to offer code that can be stuck within a non-volatile memory in the hardware. Embedded system software is intended to keep in view of the following three limits

- Convenience of system memory
- Convenience of processor's speed
- When the embedded system runs constantly, there is a necessity to limit power dissipation for actions like run, stop and wake up.

4.5.4 RTOS (Real Time Operating System)

A system which is essential to finish its task and send its service on time, then only it said to be a real time operating system. RTOS controls the application software and affords a device to allow the processor run. It is responsible for managing the different hardware resources of a personal computer and also host applications which run on the PC.

This operating system is specially designed to run various applications with an exact timing and a huge amount of consistency. Particularly, this can be significant in measurement & industrial automation systems where a delay of a program could cause a safety hazard.

4.5.5 Memory and Processors

The different kinds of processors used in an embedded system include Digital Signal Processor (DSP), microprocessor, RISC processor, microcontroller, ASSP processor, ASIP processor, and ARM processor. The different types of memories of an embedded system are given in the below chart. Each processor type serves a specific purpose based on application requirements such as speed, power efficiency, and computational complexity. For instance, DSPs handle real-time signal processing, while microcontrollers manage control-oriented tasks. RISC and ARM processors are preferred for their high performance and low power consumption. ASSP and ASIP processors are application-specific, providing optimized solutions for dedicated tasks. The memory types, as shown in the chart, play a vital role in data storage, code execution, and system performance.

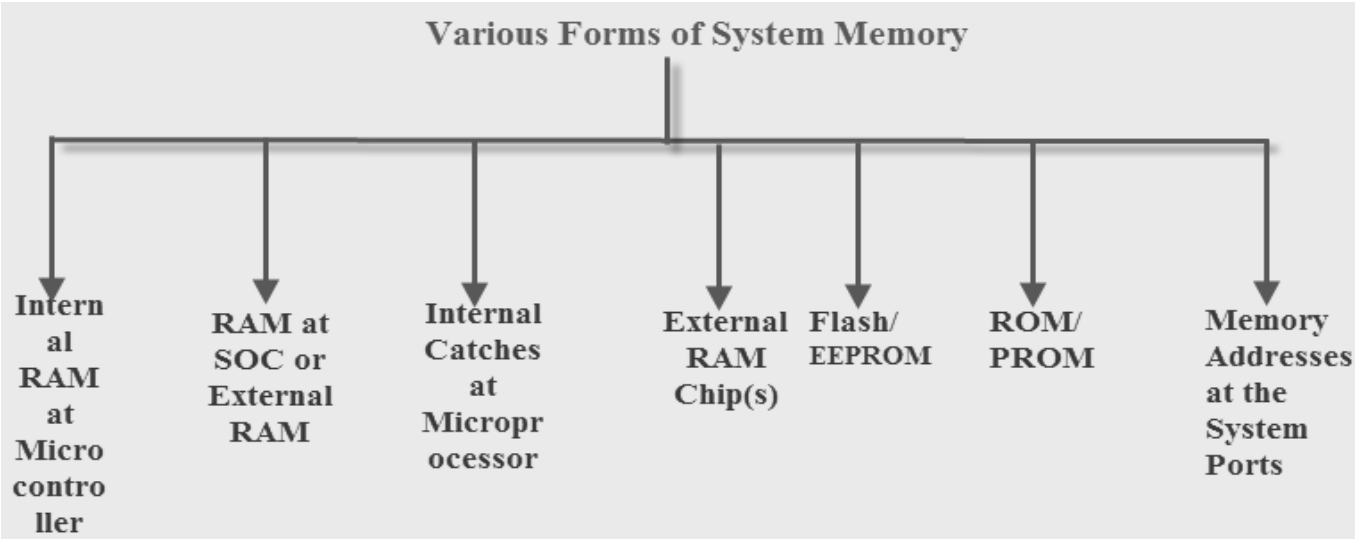

Figure 4.5.3 Various Forms of System Memory

4.5.6 Embedded System Characteristics

- Generally, an embedded system executes a particular operation and does the similar continually. For instance: A pager is constantly functioning as a pager.
- All the computing systems have limitations on design metrics, but those can be especially tight. Design metric is a measure of an execution features like size, power, cost and also performance.
- It must perform fast enough and consume less power to increase battery life.
- Several embedded systems should constantly react to changes in the system and also calculate particular results in real time without any delay. For instance, a car cruise controller; it continuously displays and responds to speed & brake sensors. It must calculate acceleration/de-accelerations frequently in a limited time; a delayed computation can consequence in letdown to control the car.
- It must be based on a microcontroller or microprocessor based.
- It must require a memory, as its software generally inserts in ROM. It does not require any secondary memories in the PC.
- It must need connected peripherals to attach input & output devices.
- An Embedded system is inbuilt with hardware and software where the hardware is used for security and performance and Software is used for more flexibility and features.

4.5.7 Embedded System Applications

The applications of an embedded system basics include smart cards, computer networking, satellites, telecommunications, digital consumer electronics, missiles, etc.

Figure 4.5.4 Embedded System

- Embedded systems in automobiles include motor control, cruise control, body safety, engine safety, robotics in an assembly line, car multimedia, car entertainment, E-com access, mobiles etc.
- Embedded systems in telecommunications include networking, mobile computing, and wireless communications, etc.
- Embedded systems in smart cards include banking, telephone and security systems.
- Embedded Systems in satellites and missiles include defense, communication, and aerospace.

4.5.8 SMS/GSM Module

The **SMS/GSM module** is another integral component of the system, responsible for **instant notifications and alerts** when abnormal fuel activity is detected. GSM (Global System for Mobile Communication) technology allows the ESP32 microcontroller to communicate with mobile networks, enabling the system to send SMS alerts to registered users regardless of their location. This ensures immediate awareness of any critical situations, such as fuel theft, leakage, or reaching the reserve level.

In the proposed system, the GSM module is connected to the ESP32 via **UART communication**, allowing the microcontroller to send AT commands that control SMS transmission. When the fuel level falls below a defined threshold or a sudden drop is detected, the ESP32 formats a message containing the relevant information, including device ID, current fuel level, and time of occurrence. This message is sent through the GSM module to the user's registered mobile number.

The main advantage of the GSM module is **reliability**. Unlike Wi-Fi-based IoT communication, which requires stable internet connectivity, the GSM module operates through mobile networks, which are widely available and dependable in urban and remote areas. This ensures that users receive **real-time**

alerts even in locations without internet access. The SMS alerts act as a backup to cloud notifications, providing redundancy in the system's monitoring capabilities.

Furthermore, the GSM module enhances **security and accountability**. Immediate alerts allow users to take prompt action against potential fuel theft or unauthorized access, minimizing financial losses and operational disruptions. Users can also receive notifications for maintenance reminders, system errors, or battery issues in the devices.

The SMS/GSM module integrates seamlessly with the IoT system, complementing remote monitoring by providing direct, actionable alerts to users' mobile devices. It is low-power, easy to interface with the ESP32, and supports multiple simultaneous alerts for fleet management applications.

In summary, the SMS/GSM module ensures **timely communication and responsiveness**, providing an effective mechanism for theft detection, reserve warnings, and overall fuel management. By combining GSM alerts with IoT cloud monitoring, the system achieves both reliability and convenience, making it suitable for vehicles, industrial tanks, and generators where real-time supervision is critical.

4.6 SOFTWARE SPECIFICATION

4.6.1 ARDUINO IDE

The Arduino integrated development environment (IDE) is a cross-platform application (for Windows, macOS, Linux) that is written in the programming language Java. It is used to write and upload programs to Arduino board.

The source code for the IDE is released under the GNU General Public License, version 2. The Arduino IDE supports the languages C and C++ using special rules of code structuring.[4] The Arduino IDE supplies a software library from the Wiring project, which provides many common input and output procedures. User-written code only requires two basic functions, for starting the sketch and the main program loop, that are compiled and linked with a program stub main() into an executable cyclic executive program with the GNU tool chain, also included with the IDE distribution. The Arduino IDE employs the program argued to convert the executable code into a text file in hexadecimal encoding that is loaded into the Arduino board by a loader program in the board's firmware.

Arduino is an open-source electronics platform based on easy-to-use hardware and software. Arduino boards are able to read inputs - light on a sensor, a finger on a button, or a Twitter

message - and turn it into an output - activating a motor, turning on an LED, publishing something online. You can tell your board what to do by sending a set of instructions to the microcontroller on the board

4.6.2 ARDUINO IOT CLOUD

Figure 4.6.1 Arduino IoT Cloud

Arduino IoT Cloud is a robust and versatile platform that enables users to develop, manage, and deploy Internet of Things (IoT) applications with ease. It is designed to integrate seamlessly with Arduino boards and other compatible hardware, allowing makers, hobbyists, and developers to create connected devices and control them remotely through a straightforward, intuitive interface. By leveraging this platform, users can build projects that gather sensor data, trigger actions, and visualize information in real time from anywhere in the world.One of the defining aspects of Arduino IoT Cloud is its user-friendly design, which lowers the barrier to entry for IoT development. It provides a visual interface where users can create things, define variables, and link devices without extensive coding knowledge. This no-code or low-code approach makes the platform accessible to beginners while still being powerful enough for advanced users who want to dive deep into custom development.The platform supports a wide range of devices, including Arduino boards like the MKR series, Nano 33 IoT, and even third-party boards compatible with the Arduino ecosystem. Users can program these boards directly from the cloud, eliminating the need for a local development environment. The Arduino Web Editor allows coding, compiling, and uploading sketches directly through a web browser, streamlining the development process and making it more flexible.With Arduino IoT Cloud, users can easily create dashboards to monitor and control their devices. These dashboards support various widgets, such as buttons, sliders, gauges, and graphs, to visualize data and interact with connected devices. The drag-and-drop interface for building dashboards means that users can quickly design custom control panels without writing complex front-end code.Security is a core consideration in Arduino IoT Cloud, with features like secure device authentication and data encryption. Devices are linked to the cloud using secure credentials, and communication is encrypted to protect sensitive information. This ensures that users can trust their connected devices to operate safely, even when accessed remotely over public networks.Arduino IoT Cloud integrates with

popular services and platforms, allowing users to build complex systems that connect with other web services and APIs. For example, users can set up automation rules to trigger actions based on specific conditions, such as turning on lights when a sensor detects motion or sending an alert when temperature thresholds are exceeded. Integrations with platforms like Amazon Alexa and Google Assistant also enable voice-controlled interactions, adding another layer of convenience.The platform includes extensive documentation, tutorials, and an active community that makes learning and troubleshooting easier. Arduino's open-source ethos means that users can access and modify source code, contribute to community projects, and share their own creations. This collaborative environment encourages innovation and continuous learning, as users can draw inspiration from one another's work and build on shared knowledge.One of the key strengths of Arduino IoT Cloud is its scalability. Users can start with simple projects, like controlling an LED or reading data from a temperature sensor, and gradually progress to more complex applications, such as home automation systems, environmental monitoring, or industrial IoT solutions. The platform's flexible structure and ability to manage multiple devices make it suitable for both small personal projects and larger deployments.The real-time data capabilities of Arduino IoT Cloud are another highlight. Users can monitor live sensor readings, track historical data, and visualize trends over time. This is invaluable for applications that rely on continuous monitoring, such as agriculture, healthcare, and smart cities. By understanding data patterns, users can make informed decisions and optimize their systems for better performance.Setting up an Arduino IoT Cloud project is straightforward. Users need to create an account, add a device, and define variables that represent the data points they want to monitor or control. The platform automatically generates code snippets to handle cloud connectivity, significantly reducing development time. Once the device is online, users can immediately start interacting with it through their custom dashboards or mobile apps.Arduino IoT Cloud also supports mobile access, enabling users to control and monitor their projects from smartphones or tablets. This adds flexibility, as users can receive push notifications, adjust settings, or check system status on the go. The mobile-friendly interface ensures a seamless experience across devices, making remote management convenient and effective.In educational settings, Arduino IoT Cloud serves as a powerful learning tool. It allows students to experiment with IoT concepts, understand real-world applications, and develop problem-solving skills. Teachers can design interactive lessons where students build connected devices, analyze data, and create their own IoT ecosystems. The platform's simplicity makes it an excellent choice for classrooms, makerspaces, and workshops.The Arduino IoT Cloud subscription model offers a free tier for hobbyists and small projects, along with paid plans for users who need more devices, extended data retention, or additional features. This flexible pricing structure ensures that users only pay for what they need, making the platform accessible to a broad audience.By providing an all-in-one solution for IoT

development, Arduino IoT Cloud empowers users to bring their ideas to life. It bridges the gap between hardware and software, making it easier than ever to create connected systems that interact with the physical world. Whether building a smart home, developing a wearable device, or prototyping an industrial solution, Arduino IoT Cloud offers the tools and support needed to succeed in the exciting world of IoT.Arduino IoT Cloud embodies the spirit of innovation and collaboration that defines the Arduino ecosystem. It fosters creativity, encourages experimentation, and helps users turn concepts into reality. With its blend of simplicity, power, and community support.

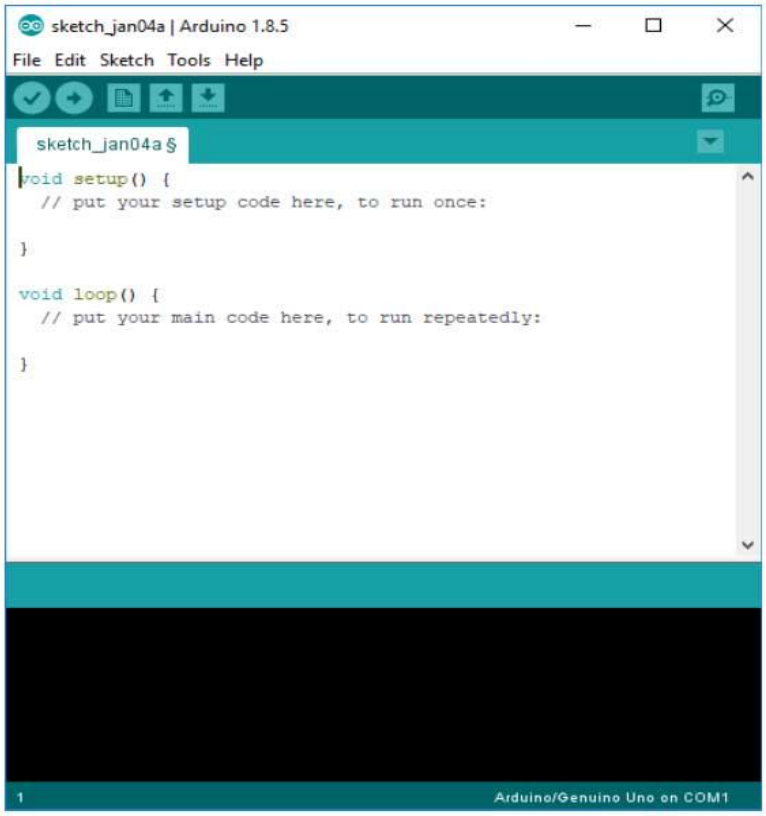

Figure 4.6.2 Arduino v1.8.5

4.6.3 THE ARDUINO IDE

The Arduino IDE is incredibly minimalistic, yet it provides a near-complete environment for most Arduino-based projects. The top menu bar has the standard options, including “File” (new, load save, etc.), “Edit” (font, copy, paste, etc.), “Sketch” (for compiling and programming), “Tools” (useful options for testing projects), and “Help”. The middle section of the IDE is a simple text editor that where you can enter the program code. The bottom section of the IDE is dedicated to an output

window that is used to see the status of the compilation, how much memory has been used, any errors that were found in the program, and various other useful messages.

Projects made using the Arduino are called sketches, and such sketches are usually written in a cut-down version of C++ (a number of C++ features are not included). Because programming a microcontroller is somewhat different from programming a computer, there are a number of device-specific libraries (e.g., changing pin modes, output data on pins, reading analog values, and timers). This sometimes confuses users who think Arduino is programmed in an "Arduino language." However, the Arduino is, in fact, programmed in C++. It just uses unique libraries for the device.

The Arduino Integrated Development Environment - or Arduino Software (IDE) - contains a text editor for writing code, a message area, a text console, a toolbar with buttons for common functions and a series of menus. It connects to the Arduino and Genuine hardware to upload programs and communicate with them.

Programs written using Arduino Software (IDE) are called **sketches**. These sketches are written in the text editor and are saved with the file extension .ino. The editor has features for cutting/pasting and for searching/replacing text. The message area gives feedback while saving and exporting and also displays errors. The console displays text output by the Arduino Software (IDE), including complete error messages and other information. The bottom right hand corner of the window displays the configured board and serial port. The toolbar buttons allow you to verify and upload programs, create, open, and save sketches, and open the serial monitor.

4.6.4 LIBRARIES

Libraries provide extra functionality for use in sketches, e.g. working with hardware or manipulating data. To use a library in a sketch, select it from the **Sketch > Import Library** menu. This will insert one or more **#include** statements at the top of the sketch and compile the library with your sketch. Because libraries are uploaded to the board with your sketch, they increase the amount of space it takes up. If a sketch no longer needs a library, simply delete its **#include** statements from the top of your code.

There is a list of libraries in the reference. Some libraries are included with the Arduino software. Others can be downloaded from a variety of sources or through the Library Manager. Starting with version 1.0.5 of the IDE, you do can import a library from a zip file and use it in an open sketch.

4.6.5 CONNECTING THE ARDUINO

Connecting an Arduino board to your PC is quite simple. On Windows:

1. Plug in the USB cable - one end to the PC, and one end to the Arduino board.

2. When prompted, select "Browse my computer for driver" and then select the folder to which you extracted your original Arduino IDE download.

3. You may receive an error that the board is not a Microsoft certified device - select “Install anyway.”

4. Your board should now be ready for programming.

5. Start to write programming

When programming your Arduino board it is important to know what COM port the Arduino is using on your PC. On Windows, navigate to Start->Devices and Printers, and look for the Arduino. The COM port will be displayed underneath.

Alternatively, the message telling you that the Arduino has been connected successfully in the lower-left hand corner of your screen usually specifies the COM port is it using.

4.6.6 PREPARING THE BOARD

Before loading any code to your Arduino board, you must first open the IDE. Double click the Arduino .exe file that you downloaded earlier. A blank program, or "sketch," should open.

The Blink example is the easiest way to test any Arduino board. Within the Arduino window, it can be found under File->Examples->Basics->Blink.

Before the code can be uploaded to your board, two important steps are required.

1. Select your Arduino from the list under Tools->Board. The standard board used in RBE 1001, 2001, and 2002 is the Arduino Mega 2560, so select the "Arduino Mega 2560 or Mega ADK" option in the dropdown.

2. Select the communication port, or COM port, by going to Tools->Serial Port.

If you noted the COM port your Arduino board is using, it should be listed in the dropdown menu. If not, your board has not finished installing or needs to be reconnected.

After selecting the correct board and COM port, click the Upload button (right arrow icon) to compile and transfer the code to your Arduino. Once the upload is complete, the onboard LED should

start blinking, confirming successful communication. This simple test ensures that both your hardware and software setups are functioning properly before moving on to more complex projects.

4.6.7 LOADING CODE

The upper left of the Arduino window has two buttons: A checkmark to Verify your code, and a right-facing arrow to Upload it. Press the right arrow button to compile and upload the Blink example to your Arduino board.

The black bar at the bottom of the Arduino window is reserved for messages indicating the success or failure of code uploading. A "Completed Successfully" message should appear once the code is done uploading to your board. If an error message appears instead, check that you selected the correct board and COM port in the Tools menu, and check your physical connections.

If uploaded successfully, the LED on your board should blink on/off once every second. Most Arduino boards have an LED prewired to pin 13.

It is very important that you do not use pins 0 or 1 while loading code. It is recommended that you do not use those pins ever.

Arduino code is loaded over a serial port to the controller. Older models use an FTDI chip which deals with all the USB specifics. Newer models have either a small AVR that mimics the FTDI chip or a built-in USB-to-serial port on the AVR micro-controller itself.

5. TESTING

System testing is a critical phase of the development process, ensuring that the **Digital Fuel Level Indicator and Reserve Alarm System** functions as intended under all expected operating conditions. Testing was performed to validate both the hardware and software components, as well as their integration, to confirm accurate fuel monitoring, real-time alerts, and cloud data updates.

5.1 UNIT TESTING

Unit testing is the first step in validating the functionality of each component within the **Digital Fuel Level Indicator and Reserve Alarm System**. It involves testing individual hardware and software modules independently to ensure that each unit performs its intended function without errors. In this system, the primary components include the **ultrasonic sensor, ESP32 microcontroller, GSM module, and LCD display**.

For the ultrasonic sensor, unit testing focused on measuring distances accurately. The sensor was placed at various fuel levels in the tank, and readings were compared with actual measurements to ensure precision. This step is crucial because the entire system's reliability depends on the accuracy of fuel level detection. Similarly, the ESP32 microcontroller was tested for correct data processing. It was programmed with test inputs to verify that threshold detection, fuel level calculation, and alert logic were functioning properly.

The **GSM module** was evaluated for its ability to send SMS notifications upon receiving a command from the ESP32. Test messages were sent to registered numbers to confirm reliability and speed of delivery. The LCD display unit was also tested to ensure that it correctly visualized fuel levels, reserve status, and alert notifications. Each of these tests ensured that the individual components could perform their respective tasks independently before integration.

Unit testing not only verifies the correct operation of hardware and software but also identifies potential issues early in the development process. Any malfunction or inconsistency can be resolved at this stage, reducing the risk of system failures after full integration. By ensuring that each component meets the required specifications, unit testing provides a solid foundation for building a robust and reliable

fuel monitoring system. It guarantees that when the modules are integrated, the overall system will function seamlessly and accurately, providing precise fuel management and timely alerts.

5.2 INTEGRATION TESTING

Integration testing focuses on validating the interaction between multiple components after unit testing has been completed. In the **Digital Fuel Level Indicator and Reserve Alarm System**, integration testing ensures that the **ultrasonic sensor, ESP32, GSM module, LCD display, and IoT cloud platform** work together cohesively.

During integration testing, the ultrasonic sensor readings were fed to the ESP32 microcontroller, which processed the data according to programmed thresholds. The ESP32 then triggered the GSM module to send SMS alerts whenever abnormal fuel drops or reserve-level conditions were detected. Simultaneously, fuel level data was sent to the IoT cloud dashboard, allowing remote monitoring. This step was essential to confirm that signals and data flows between components were accurate, timely, and synchronized.

The LCD display was also integrated into the testing phase to validate that local visualization matched the real-time readings and threshold-based alerts. The cloud dashboard received continuous updates, reflecting the same readings displayed on the LCD. Integration testing involved simulating various scenarios, including normal consumption, sudden fuel drops, and reaching the reserve level, to ensure the system reacted correctly in all cases.

Integration testing identifies compatibility issues, communication delays, and logical errors between components. For instance, it helped verify that the GSM module responded promptly to ESP32 commands and that the IoT dashboard displayed accurate information without lag. This testing phase ensures that the system functions as a unified solution, providing **real-time fuel monitoring, prompt alerts, and reliable historical data logging**. By confirming component interoperability, integration testing guarantees that the system is robust, efficient, and ready for real-world deployment.

5.3 FUNCTIONAL TESTING

Functional testing evaluates whether the **Digital Fuel Level Indicator and Reserve Alarm System** meets the operational requirements specified during the design phase. It ensures that the system performs

all intended functions accurately, including **fuel level measurement, reserve alerts, theft detection, SMS notifications, LCD visualization, and cloud updates**.

In functional testing, the system was subjected to realistic scenarios, such as normal fuel consumption, rapid fuel withdrawal, and tank refilling. The ultrasonic sensor continuously measured fuel levels and transmitted this data to the ESP32, which processed it to detect abnormal changes. When fuel levels dropped suddenly or reached the reserve threshold, the ESP32 triggered both the GSM module and IoT cloud to issue alerts. Functional testing verified that these alerts were accurate, timely, and corresponded to the actual fuel conditions.

The LCD display and cloud dashboard were also tested to confirm that they provided consistent and real-time information. Graphs and indicators on the cloud interface were checked for correct updates, while the SMS notifications were examined for accuracy, readability, and prompt delivery. This ensured that the system was fully functional and reliable in monitoring and alerting users.

Functional testing is critical because it validates **the system's capability to meet its primary objectives**. It ensures that fuel theft and leakage detection, reserve warnings, and remote monitoring functions operate seamlessly under various conditions. By simulating real-world operations, functional testing guarantees that the system delivers precise, actionable, and reliable information, reducing fuel loss, enhancing security, and improving operational efficiency.

6. CONCLUSION

The **Digital Fuel Level Indicator and Reserve Alarm System** presents an effective, reliable, and technologically advanced solution for monitoring fuel levels in vehicles, generators, and industrial tanks. By integrating **ultrasonic sensing, ESP32 microcontroller processing, GSM communication, and IoT cloud connectivity**, the system provides real-time fuel monitoring, accurate detection of abnormal fuel consumption, and prompt alerts to users, significantly reducing the risks of fuel theft, leakage, and unauthorized usage.

Experimental testing and validation have demonstrated the system's ability to continuously measure fuel levels, process readings intelligently, and generate timely notifications both locally via LCD display and remotely through SMS and cloud dashboards. The IoT integration enables remote access and historical data analysis, empowering users to make informed decisions, optimize fuel management, and maintain operational efficiency. Additionally, the system's automated alerts provide an extra layer of security and accountability, ensuring that potential losses are minimized.

One of the key advantages of this system is its **scalability and ease of implementation**. It can be deployed in a single vehicle, an industrial fuel tank, or a fleet of vehicles without significant modifications, making it versatile and cost-effective. Moreover, by replacing manual fuel monitoring methods, it eliminates human errors and provides precise, actionable information in real time.

In conclusion, the proposed system successfully addresses the limitations of traditional fuel monitoring approaches. It enhances security, improves resource management, and ensures efficient fuel utilization through automation and intelligent monitoring. The combination of IoT technology, GSM alerts, and embedded systems creates a comprehensive solution that is practical, reliable, and adaptable for various real-world applications, ultimately contributing to economic savings and improved operational control.

7.REFERENCES

1. D. Sur, J. Firdaus, R. Dutta, A. Purohit, D. Mitra, and K. Majumder, "IoT based fuel level monitoring and fuel theft detection system," International Journal of Scientific Research in Engineering and Management (IJSREM), vol. 4, no. 2, pp. 1–5, Feb. 2020. [Online]. Available: https://doi.org/10.36375/prepare_u.a87
2. R. Bose, S. Roy, S. Chakraborty, and I. Sarkar, "Development of a real-time fuel monitoring system for construction industry using Internet of Things," Journal of Scientific Research and Reports, vol. 30, no. 11, pp. 1064–1070, Nov. 2024. [Online]. Available: https://doi.org/10.9734/jsrr/2024/v30i112633
3. B. Anand, K. H. S. Kathyayini, and A. Kumar, "An IoT-based fuel monitoring and tracking system with real-time fuel level detection using accelerometer," Journal of Scientific Research and Reports, vol. 30, no. 11, pp. 1064–1070, Nov. 2024. [Online]. Available: https://doi.org/10.9734/jsrr/2024/v30i112633
4. S. Prabhu, D. Kaliyaperumal, and M. Nithya, "IoT enabled fuel level monitoring and automatic fuel theft detection system," in Proc. 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), Bangalore, India, 2022, pp. 1–5. [Online]. Available: https://doi.org/10.1109/ICCCNT54827.2022.9984515
5. M. Rakesh, "An IoT-based fuel monitoring and tracking system with real-time fuel level detection using accelerometer," Journal of Scientific Research and Reports, vol. 30, no. 11, pp. 1064–1070, Nov. 2024. [Online]. Available: https://doi.org/10.9734/jsrr/2024/v30i112633
6. B. Anand, K. H. S. Kathyayini, and A. Kumar, "An IoT-based fuel monitoring and tracking system with real-time fuel level detection using accelerometer," Journal of Scientific Research and Reports, vol. 30, no. 11, pp. 1064–1070, Nov. 2024. [Online]. Available: https://doi.org/10.9734/jsrr/2024/v30i112633
7. S. Prabhu, D. Kaliyaperumal, and M. Nithya, "IoT enabled fuel level monitoring and automatic fuel theft detection system," in Proc. 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), Bangalore, India, 2022, pp. 1–5. [Online]. Available: https://doi.org/10.1109/ICCCNT54827.2022.9984515
8. M. Rakesh, "An IoT-based fuel monitoring and tracking system with real-time fuel level detection using accelerometer," Journal of Scientific Research and Reports, vol. 30, no. 11, pp. 1064–1070, Nov. 2024. [Online]. Available: https://doi.org/10.9734/jsrr/2024/v30i112633

9. S. Prabhu, D. Kaliyaperumal, and M. Nithya, "IoT enabled fuel level monitoring and automatic fuel theft detection system," in Proc. 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), Bangalore, India, 2022, pp. 1–5. [Online]. Available: https://doi.org/10.1109/ICCCNT54827.2022.9984515
10. M. Rakesh, "An IoT-based fuel monitoring and tracking system with real-time fuel level detection using accelerometer," Journal of Scientific Research and Reports, vol. 30, no. 11, pp. 1064–1070, Nov. 2024. [Online]. Available: https://doi.org/10.9734/jsrr/2024/v30i112633
11. S. Prabhu, D. Kaliyaperumal, and M. Nithya, "IoT enabled fuel level monitoring and automatic fuel theft detection system," in Proc. 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), Bangalore, India, 2022, pp. 1–5. [Online]. Available: https://doi.org/10.1109/ICCCNT54827.2022.9984515
12. M. Rakesh, "An IoT-based fuel monitoring and tracking system with real-time fuel level detection using accelerometer," Journal of Scientific Research and Reports, vol. 30, no. 11, pp. 1064–1070, Nov. 2024. [Online]. Available: https://doi.org/10.9734/jsrr/2024/v30i112633
13. S. Prabhu, D. Kaliyaperumal, and M. Nithya, "IoT enabled fuel level monitoring and automatic fuel theft detection system," in Proc. 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), Bangalore, India, 2022, pp. 1–5. [Online]. Available: https://doi.org/10.1109/ICCCNT54827.2022.9984515
14. M. Rakesh, "An IoT-based fuel monitoring and tracking system with real-time fuel level detection using accelerometer," Journal of Scientific Research and Reports, vol. 30, no. 11, pp. 1064–1070, Nov. 2024. [Online]. Available: https://doi.org/10.9734/jsrr/2024/v30i112633
15. S. Prabhu, D. Kaliyaperumal, and M. Nithya, "IoT enabled fuel level monitoring and automatic fuel theft detection system," in Proc. 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), Bangalore, India, 2022, pp. 1–5. [Online]. Available: https://doi.org/10.1109/ICCCNT54827.2022.9984515
16. M. Rakesh, "An IoT-based fuel monitoring and tracking system with real-time fuel level detection using accelerometer," Journal of Scientific Research and Reports, vol. 30, no. 11, pp. 1064–1070, Nov. 2024. [Online]. Available: https://doi.org/10.9734/jsrr/2024/v30i112633
17. S. Prabhu, D. Kaliyaperumal, and M. Nithya, "IoT enabled fuel level monitoring and automatic fuel theft detection system," in Proc. 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), Bangalore, India, 2022, pp. 1–5. [Online]. Available: https://doi.org/10.1109/ICCCNT54827.2022.9984515

18. M. Rakesh, "An IoT-based fuel monitoring and tracking system with real-time fuel level detection using accelerometer," Journal of Scientific Research and Reports, vol. 30, no. 11, pp. 1064–1070, Nov. 2024. [Online]. Available: https://doi.org/10.9734/jsrr/2024/v30i112633
19. S. Prabhu, D. Kaliyaperumal, and M. Nithya, "IoT enabled fuel level monitoring and automatic fuel theft detection system," in Proc. 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), Bangalore, India, 2022, pp. 1–5. [Online]. Available: https://doi.org/10.1109/ICCCNT54827.2022.9984515
20. M. Rakesh, "An IoT-based fuel monitoring and tracking system with real-time fuel level detection using accelerometer," Journal of Scientific Research and Reports, vol. 30, no. 11, pp. 1064–1070, Nov. 2024. [Online]. Available: https://doi.org/10.9734/jsrr/2024/v30i112633
21. S. Prabhu, D. Kaliyaperumal, and M. Nithya, "IoT enabled fuel level monitoring and automatic fuel theft detection system," in Proc. 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), Bangalore, India, 2022, pp. 1–5. [Online]. Available: https://doi.org/10.1109/ICCCNT54827.2022.9984515
22. M. Rakesh, "An IoT-based fuel monitoring and tracking system with real-time fuel level detection using accelerometer," Journal of Scientific Research and Reports, vol. 30, no. 11, pp. 1064–1070, Nov. 2024. [Online]. Available: https://doi.org/10.9734/jsrr/2024/v30i112633
23. S. Prabhu, D. Kaliyaperumal, and M. Nithya, "IoT enabled fuel level monitoring and automatic fuel theft detection system," in Proc. 13th International Conference on Computing Communication and Networking Technologies (ICCCNT), Bangalore, India, 2022, pp. 1–5. [Online]. Available: https://doi.org/10.1109/ICCCNT54827.2022.9984515
24. M. Rakesh, "An IoT-based fuel monitoring and tracking system with real-time fuel level detection using accelerometer," Journal of Scientific Research and Reports, vol. 30.

8. APPENDIX

8.1 SOURCE CODE

```
#include <LiquidCrystal.h>

// ========== LCD Pins (RS, E, D4, D5, D6, D7) ==========
LiquidCrystal lcd(2, 15, 19, 18, 5, 4);

#define TRIG_PIN 27
#define ECHO_PIN 26

// ---------------- Tank calibration ----------------
const float TANK_HEIGHT_CM = 30.0;   // Height of tank in cm

// ---------------- Parameters ----------------
const int RAW_SAMPLES = 5;
const unsigned long MEASUREMENT_INTERVAL = 2000; // ms
const float SUDDEN_DROP_THRESHOLD = 10.0;        // % drop considered theft

// ---------------- State ----------------
float prevLevel = 100.0;
unsigned long lastMeasure = 0;

// ---------------- Functions ----------------
float readMedianDistanceCm() {
  float samples[RAW_SAMPLES];
  for (int i = 0; i < RAW_SAMPLES; i++) {
    digitalWrite(TRIG_PIN, LOW);
    delayMicroseconds(2);
```

```
    digitalWrite(TRIG_PIN, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG_PIN, LOW);

    unsigned long dur = pulseIn(ECHO_PIN, HIGH, 30000UL); // timeout 30ms
    float dist = (dur > 0) ? (dur * 0.034f) / 2.0f : TANK_HEIGHT_CM;

    samples[i] = dist;
    delay(50);
  }

  // sort
  for (int i = 0; i < RAW_SAMPLES - 1; i++) {
    for (int j = i + 1; j < RAW_SAMPLES; j++) {
      if (samples[j] < samples[i]) {
        float tmp = samples[i]; samples[i] = samples[j]; samples[j] = tmp;
      }
    }
  }
  return samples[RAW_SAMPLES / 2];
}

float distanceToPercent(float distanceCm) {
  float pct = (1.0 - (distanceCm / TANK_HEIGHT_CM)) * 100.0;
  if (pct < 0) pct = 0;
  if (pct > 100) pct = 100;
  return pct;
}

// ---------------- Setup ----------------
```

```
void setup() {
  Serial.begin(9600);
  lcd.begin(16, 2);
  lcd.print("Fuel Monitor...");

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  delay(1000);
  lcd.clear();
}

// ---------------- Loop ----------------
void loop() {
  unsigned long now = millis();
  if (now - lastMeasure >= MEASUREMENT_INTERVAL) {
    lastMeasure = now;

    float dist = readMedianDistanceCm();
    float level = distanceToPercent(dist);

    String status;
    if (level < prevLevel)
    {
      float drop = prevLevel - level;
      if (drop >= SUDDEN_DROP_THRESHOLD)
      {
        status = "Theft/Leakage!";
        gsm("Petrol Theft");   //  Send SMS when theft detected
      }
      else
```

```
  {
    status = "Gradual Reduce";
  }
}
else if (level > prevLevel)
{
  status = "Fuel Added";
}
else
{
  status = "Stable";
}

// --- Display on LCD ---
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Fuel: ");
lcd.print(level, 1);
lcd.print("%");

lcd.setCursor(0, 1);
lcd.print(status);

// --- Debug Serial ---
Serial.print("Raw distance: ");
Serial.print(dist, 1);
Serial.print(" cm | Fuel: ");
Serial.print(level, 1);
Serial.print("% | Status: ");
Serial.println(status);
```

```
    prevLevel = level;
  }
}

/// --- GSM Function ---
void gsm(String a)
{
  Serial.println("AT");              // Send AT command
  delay(100);

  Serial.println("AT+CMGF=1");        // Set SMS mode to text
  delay(100);

  Serial.print("AT+CMGS=\"9159883566\"\r"); // your number
  delay(100);

  Serial.print(a);              // SMS content
  delay(100);

  Serial.write(0x1A);                // End SMS (CTRL+Z)
  delay(100);
}
```

8.2 OUTPUT

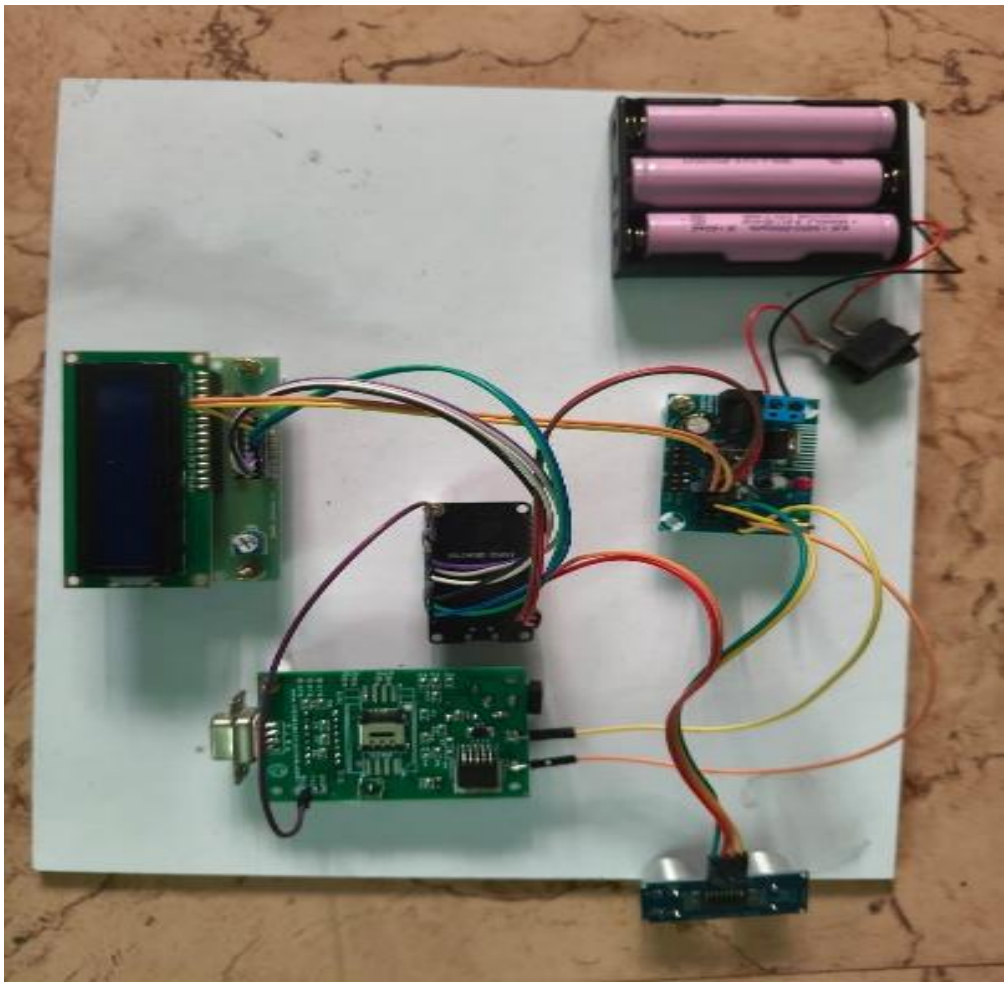

Figure 8.2.1 Circuit Board

Figure 8.2.2 Theft Alert

Figure 8.2.3 Fuel Added

Figure 8.2.4 Gradual Reduce

Figure 8.2.5 Stable